A Reproducible Computer Vision Study Across Fine-Grained Recognition, Pet Segmentation, and Aerial Object Detection

Yuheng Li
Student ID: 23307130334
GitHub: https://github.com/yuheng-li-ai/CV_Midterm
Model weights: ModelScope repository

Abstract

This report presents a unified experimental study for three computer vision tasks: Oxford-IIIT Pet image classification, Oxford-IIIT Pet semantic segmentation, and VisDrone aerial object detection with video multi-object tracking. A configuration-driven harness was implemented before large-scale training, so that all models, losses, metrics, checkpoints, and visualizations follow the same reproducible convention. For classification, ImageNet pretraining was the dominant factor, improving ResNet-18 test accuracy from 40.15% to about 89.40%, while the best extension was a pretrained ResNet-34 with 90.60% test accuracy. For segmentation, a from-scratch U-Net trained with Dice loss achieved the best required result, reaching 77.90% test mIoU. For detection, YOLO11m at 960-pixel input resolution substantially improved VisDrone mAP50-95 over YOLOv8n and YOLOv8s, and the resulting detector was used for tracking, occlusion analysis, and line-crossing counting on a 30-second rainy traffic video. Beyond reporting scores, the analysis emphasizes negative results, failure cases, and the relationship between metric gains and qualitative behavior.

1 Introduction

The assignment contains three tasks that stress different levels of visual understanding. Fine-grained pet classification tests instance-level appearance recognition. Pet semantic segmentation tests dense pixel-level localization. VisDrone detection and tracking test small-object perception in crowded aerial or high-view scenes. Treating these tasks as isolated scripts would make the experiments difficult to audit, so I first built a unified harness around configuration files, fixed output directories, checkpointing, CSV logging, curve generation, and report-ready visualization.

The main goal of this report is not only to obtain competitive numbers under the course setting, but also to explain why certain design choices worked or failed. Therefore, every required experiment is paired with at least one extension: pretraining and architecture capacity for classification, loss engineering for segmentation, and detector capacity, image resolution, tracker choice, and confidence threshold for detection and tracking.

2 Unified Experimental Harness

The project is organized around a single Python package, `cvhw2`. The command-line interface exposes five major commands: classification training, segmentation training, VisDrone conversion, detection training, and video tracking. Configurations are stored in `configs/`; scalar logs are written to `runs/`; checkpoints are written to `checkpoints/`; and final visual evidence is collected in `assets/`. This structure was useful because the assignment required many comparisons, and accidental changes in data loading, evaluation split, or checkpoint selection would otherwise make the results hard to interpret.

The released final and auxiliary checkpoints are hosted on ModelScope at https://www.modelscope.cn/models/yuhengli/cv-hw2-final-model. The ModelScope repository includes the three final models, the comparison checkpoints used in the report, the training configurations, and this final PDF.

The most important harness rule is validation-based model selection. During training, the best checkpoint is selected by validation accuracy for classification, validation mIoU for segmentation, and validation mAP50-95 for detection. The final test result is then computed from that best checkpoint instead of from the last epoch. This matters because several curves peaked before the final epoch, especially the pretrained ResNet-18 classification baseline and some segmentation losses. The harness also automatically saves learning curves and qualitative outputs, which prevents the final report from relying only on scalar metrics.

3 Task 1: Pet Image Classification

3.1 Dataset and Protocol

The classification task uses the Oxford-IIIT Pet dataset with 37 pet categories. Images are resized to 224×224 for full experiments. Training uses the official trainval portion with an internal validation split, and final evaluation uses the test split. The metric is top-1 accuracy. All main experiments use AdamW with weight decay 10^{-4} and a 20-epoch budget. For pretrained models, the backbone learning rate is 10^{-4} and the newly initialized classifier head learning rate is 10^{-3}. This split learning-rate design reflects the assumption that generic

ImageNet features should be adapted gently, while the task-specific 37-way classifier must learn faster.

3.2 Required Models

The required baseline is an ImageNet-pretrained ResNet-18 with a replaced final classifier. The pretraining ablation uses the same architecture but random initialization. The attention extension inserts a squeeze-and-excitation module after the final residual stage of a pretrained ResNet-18. These three experiments isolate three questions: whether transfer learning is necessary, whether the baseline is already strong, and whether a lightweight channel-attention module adds value.

Table 1: Classification results on Oxford-IIIT Pet.

Experiment	Best Val	Test Acc.	Note
ResNet-18 pretrained	0.9185	0.8940	required
ResNet-18 random	0.4475	0.4015	required
SE-ResNet-18	0.9112	0.8934	required
ResNet-34 pretrained	0.9330	0.9060	extension
Label smoothing	0.9130	0.8970	extension
Freeze then unfreeze	0.9239	0.8986	extension

3.3 Classification Results and Discussion

Table 1 shows that ImageNet pretraining is the strongest factor in this task. The pretrained ResNet-18 reached 89.40% test accuracy, while the same architecture trained from scratch reached only 40.15% under the same 20-epoch budget. The gap is too large to be explained by random variation. It indicates that generic low-level and mid-level features learned from ImageNet transfer well to pet breed recognition, especially when the target dataset is not large enough to train a deep residual network from scratch efficiently.

The random model was still improving at epoch 20, so the comparison should not be interpreted as proof that random initialization can never work. Rather, it shows that pretraining is vastly more sample- and compute-efficient under the assignment budget. This distinction is important: the ablation answers a practical question about the chosen training regime, not a universal theoretical question.

The SE-ResNet-18 result is a useful negative result. Although channel attention is often expected to help fine-grained recognition, the SE variant slightly underperformed the simpler pretrained ResNet-18. One plausible explanation is that the pretrained ResNet-18 already contains strong channel representations, while the newly inserted SE block introduces extra randomly initialized parameters that must be learned from limited data. The result does not invalidate attention mechanisms in general, but it warns against assuming that a more complex module will automatically improve a strong pretrained baseline.

Among the extensions, the pretrained ResNet-34 achieved the best test accuracy, 90.60%. This suggests that moderate additional depth can improve fine-grained discrimination when pretrained features are available. Label smoothing and freeze-then-unfreeze gave smaller gains over the baseline. Label smoothing slightly improved test accuracy but lowered best validation accuracy, which suggests mild regularization rather than a decisive improvement. Freeze-then-unfreeze improved validation accuracy and test accuracy modestly, supporting the idea that stabilizing the new classifier head before full fine-tuning can reduce early optimization noise.

3.4 Classification Hyperparameter Grid Search

To address the hyperparameter-analysis requirement explicitly, I added a controlled grid search around the pretrained ResNet-18 baseline. The model architecture, data split, input size, batch size, backbone learning rate, and weight decay were held fixed. The search varied two optimization hyperparameters: the classifier-head learning rate and label smoothing. Each setting was trained for 8 epochs as a budgeted search, and validation accuracy was used as the selection criterion. The final test split was not used to choose the hyperparameter setting.

Table 2: Budgeted classification hyperparameter grid search. The best validation setting is highlighted.

Label smoothing	Head LR	Best epoch	Best Val	Test Acc.
0.00	5×10^{-4}	3	0.9112	0.8803
0.00	1×10^{-3}	3	0.9130	0.8760
0.00	2×10^{-3}	5	0.9130	0.8880
0.05	5×10^{-4}	5	0.9058	0.8863
0.05	1×10^{-3}	**5**	**0.9167**	**0.8880**
0.05	2×10^{-3}	7	0.9094	0.8926
0.10	5×10^{-4}	7	0.9076	0.8872
0.10	1×10^{-3}	6	0.9130	0.8902
0.10	2×10^{-3}	7	0.9076	0.8937

The grid search shows that the classification result is not highly sensitive to these two hyperparameters within the tested range. The best validation result, 0.9167, occurs at label smoothing 0.05 and head learning rate 10^{-3}, which is close to the original baseline design. Increasing label smoothing to 0.10 does not improve validation accuracy, suggesting that too much target softening can weaken the fine-grained breed signal. A larger head learning rate improves some test numbers but does not consistently improve validation performance, so I did not use it to redefine the final model. This supports a conservative conclusion: careful fine-tuning matters, but pretraining and backbone capacity remain the dominant factors in this task.

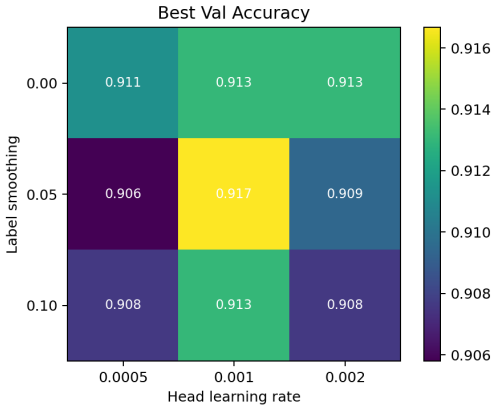


Figure 1: Validation accuracy heatmap for the classification hyperparameter grid. The narrow range of values indicates mild sensitivity compared with the much larger effect of ImageNet pretraining.

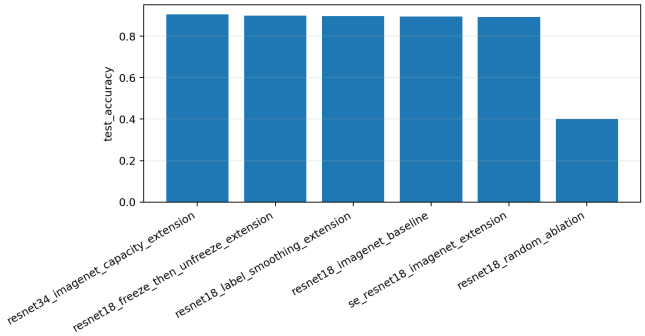


Figure 2: Classification test accuracy comparison. The main gain comes from ImageNet pretraining, while the best extension is the larger pretrained ResNet-34.

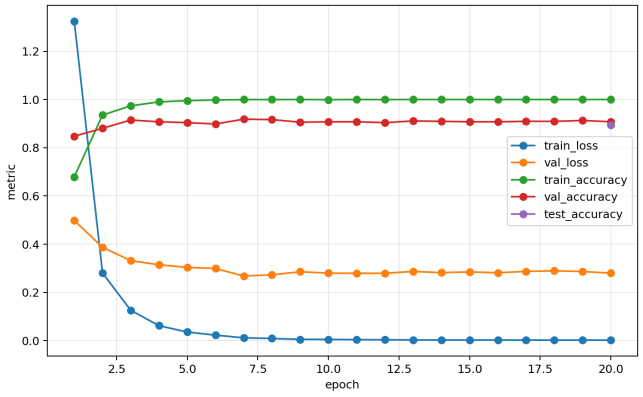


Figure 3: Training curves for the ImageNet-pretrained ResNet-18 baseline. The validation peak occurs early, showing why best-checkpoint evaluation is

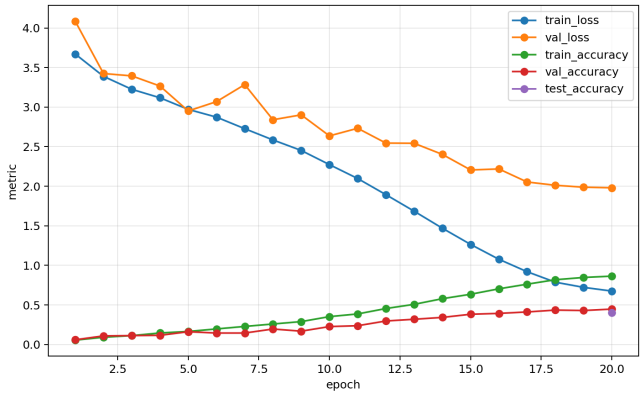


Figure 4: Training curves for the randomly initialized ResNet-18. Compared with Fig. 3, convergence is much slower and the validation accuracy remains far below the pretrained model under the same epoch budget.

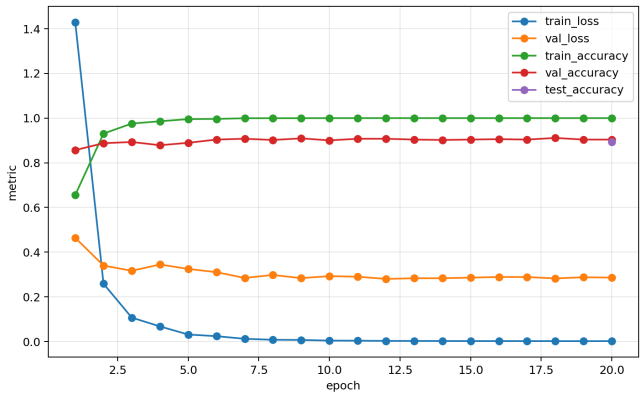


Figure 5: Training curves for the SE-ResNet-18 extension. The curve shows that attention did not create a clear generalization gain over the simpler pretrained ResNet-18 baseline.

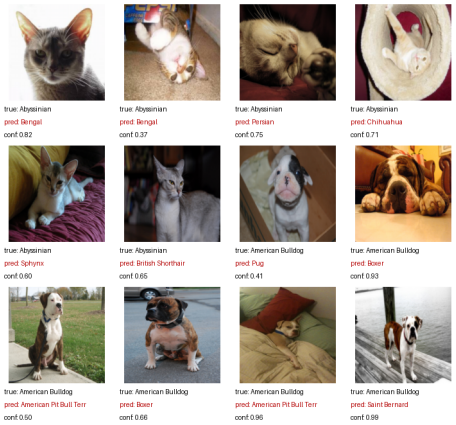
3.5 Error Analysis

The confusion matrix in Fig. 6 and the mistake grid in Fig. 7 show that most errors are semantically plausible rather than arbitrary. The top confusion pairs include American Pit Bull Terrier versus Staffordshire Bull Terrier, Staffordshire Bull Terrier versus American Pit Bull Terrier, Ragdoll versus Birman, American Pit Bull Terrier versus American Bulldog, and Egyptian Mau versus Bengal. These are visually similar classes with shared body shape, fur color, or facial structure. The model therefore fails mainly in fine-grained discrimination, not in coarse pet recognition.

This observation changes how the classification result should be judged. A test accuracy near 90% is strong, but the remaining errors are concentrated exactly where subtle anatomical cues are required. More data augmentation, higher-resolution crops, or part-aware features might help these cases more than a generic increase in classifier capac-

Interpretation. The matrix is dominated by a strong diagonal, which is consistent with the near-90% test accuracy. The visible off-diagonal

Figure 6: Confusion matrix for the pretrained ResNet-18 baseline with adjacent interpretation. The strongest off-diagonal blocks correspond to visually similar breeds rather than random category failures.



Qualitative error pattern. The representative mistakes show that background clutter is not the only source of failure. Several wrong predictions involve animals photographed in clear view, but the discriminative breed cues are subtle. The pit bull, Staffordshire bull terrier, and bulldog group is difficult because of similar short fur and head structure. The Ragdoll, Birman, and Siamese group is difficult because coat color and facial color masks overlap. These examples explain why the ResNet-34 capacity extension helps but does not eliminate the hardest errors: more capacity improves representation, but the dataset still asks for local anatomical distinctions.

Figure 7: Representative classification mistakes from the pretrained ResNet-18 baseline. A compact subset is shown to keep the visual evidence close to the discussion without creating a page-breaking tall grid.

4 Task 3: Pet Semantic Segmentation

4.1 Dataset, Model, and Losses

The segmentation task also uses Oxford-IIIT Pet, but with trimap masks converted into a three-class semantic segmentation problem: pet foreground, boundary, and background. The model is a manually implemented U-Net trained from random initialization. It contains four encoder stages, a bottleneck, four decoder stages, transposed-convolution upsampling, and skip concatenations. The main metric is mean Intersection over Union (mIoU).

Three losses form the required comparison. Cross-entropy treats each pixel independently and is stable early in training. Multiclass Dice loss optimizes region overlap more directly. The combined CE+Dice objective attempts to balance stable pixel-wise supervision with overlap-oriented optimization. For a class c, the Dice term is computed as

$$\mathcal{L}_{Dice} = 1 - \frac{1}{C}\sum_{c=1}^{C}\frac{2\sum_i p_{i,c}y_{i,c} + s}{\sum_i p_{i,c} + \sum_i y_{i,c} + s}, \qquad (1)$$

where $p_{i,c}$ is the predicted probability, $y_{i,c}$ is the one-hot target, $C = 3$, and s is the smoothing constant. All main segmentation experiments use image size 224, batch size 16, AdamW, cosine learning-rate scheduling, and a 60-epoch budget.

4.2 Required Loss Comparison

Table 3: Segmentation results for the 60-epoch U-Net loss comparison.

Loss	Best Epoch	Best Val mIoU	Test mIoU
Cross-Entropy	42	0.7640	0.7640
Dice	54	0.7748	0.7790
CE + Dice	37	0.7681	0.7741

Dice loss achieved the best required result, with 77.90% test mIoU. This supports the hypothesis that overlap-oriented supervision is better aligned with the mIoU metric than pure pixel-wise cross-entropy. Cross-entropy remained a strong baseline, but it peaked earlier and did not match the final overlap quality of Dice. CE+Dice improved over CE but did not surpass Dice alone, suggesting that the stabilizing CE term was not necessary once the training budget was long enough for Dice optimization to become stable.

The interpretation should remain bounded. Dice was best in this controlled setup, but this does not mean Dice is universally superior. Cross-entropy can be easier to optimize early, and Dice can be sensitive when classes are extremely small or predictions are unstable. In this dataset and schedule, however, the object-overlap objective matched the evaluation target well.

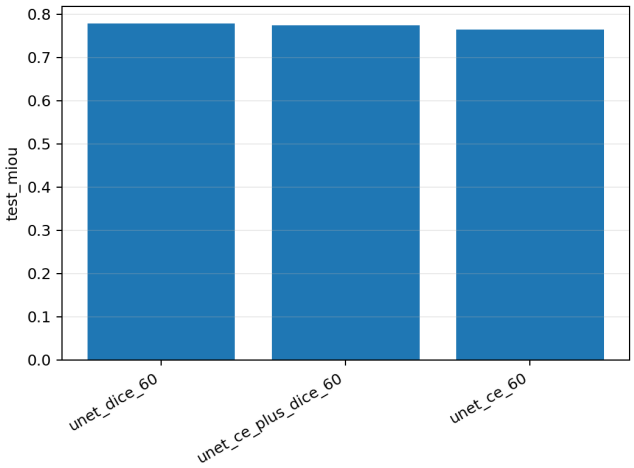


Figure 8: Test mIoU for the required 60-epoch segmentation losses. Dice loss gives the strongest required result.

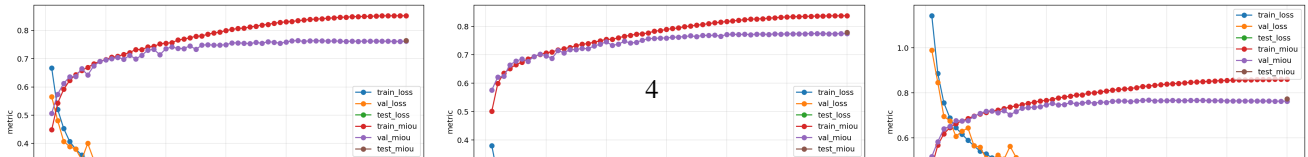


Figure 9: Training curves for the required segmentation losses: CE, Dice, and CE+Dice from left to right. These panels have the same aspect ratio and are therefore compared side by side. Dice improves more slowly but reaches the best validation and test mIoU.

4.3 Loss Engineering Extensions

Two additional loss experiments were conducted. First, the Dice smoothing constant was reduced from 1.0 to 0.1. This slightly increased best validation mIoU from 0.7748 to 0.7769, but the test mIoU was almost unchanged, 0.7785 versus 0.7790. The smaller smoothing value may make the objective more sensitive to foreground and boundary errors, but the difference is too small to claim a robust generalization improvement.

Second, Focal Loss was tested as a hard-pixel emphasis strategy. It reached 0.7645 test mIoU, close to cross-entropy but below Dice. This is another useful negative result. Boundary pixels in trimap segmentation are hard not only because they are informative, but also because they are ambiguous and sometimes visually noisy. Emphasizing all hard pixels can therefore over-focus the model on uncertain regions instead of improving structural mask quality.

Table 4: Segmentation extension results.

Experiment	Best Epoch	Best Val mIoU	Test mIoU
Dice, smooth 1.0	54	0.7748	0.7790
Dice, smooth 0.1	55	0.7769	0.7785
Focal Loss	45	0.7637	0.7645

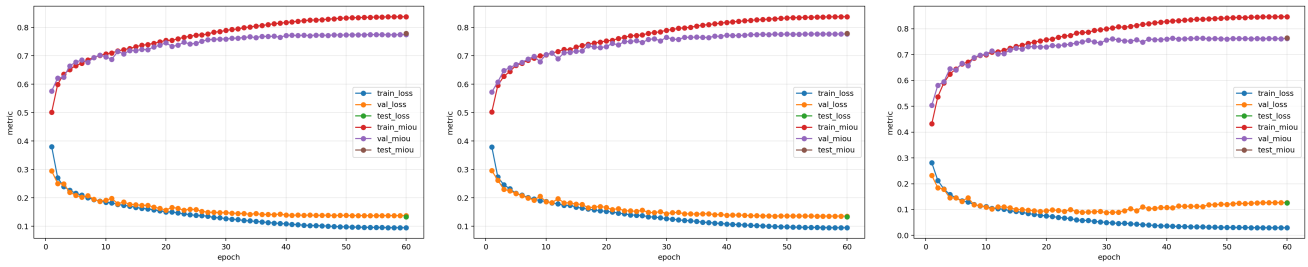
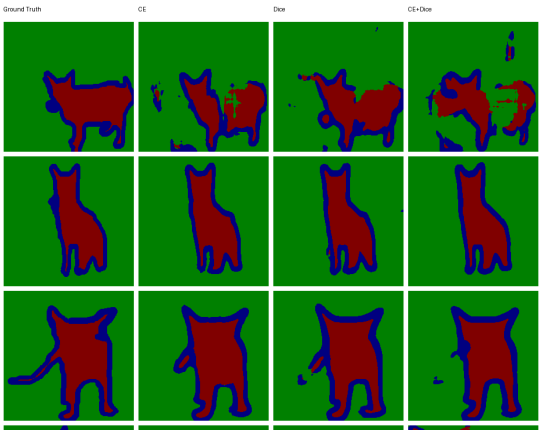


Figure 10: Training curves for segmentation extensions: Dice with smooth 1.0, Dice with smooth 0.1, and Focal Loss from left to right. The two Dice variants are close, while Focal Loss does not improve mIoU despite emphasizing hard pixels.

4.4 Qualitative Segmentation Analysis

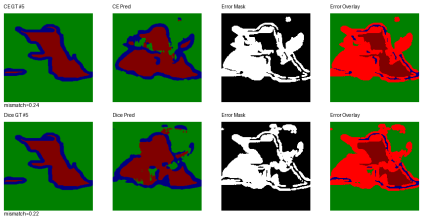
Figure 12 compares ground-truth masks, predictions, binary error maps, and overlays for representative difficult examples. The dominant errors are not complete object misses; they are boundary and contour errors. Thin legs, ears, fur edges, and low-contrast foreground-background transitions are harder than the main body region. This explains why mIoU can improve while the masks still look imperfect around fine structures.

This qualitative evidence also clarifies the Focal Loss result. If the difficult pixels are mainly ambiguous boundaries, then increasing their weight does not necessarily create cleaner boundaries. It may instead amplify label noise or local uncertainty. Dice improves global overlap, but even it does not fully solve contour precision. A natural next step would be a boundary-aware loss or higher-resolution decoding, but that is beyond the required scope.



Mask-level reading. The qualitative masks confirm the quantitative ordering in Table 3, but they also show why mIoU is not the whole story. Dice improves global object overlap, while CE is stable but less directly aligned with overlap. The remaining errors concentrate on ears, legs, fur boundaries, and regions where the pet and background have similar color. Therefore, the best Dice score should be understood as better region consistency rather than perfect contour recovery.

Figure 11: Compact qualitative mask examples for the required segmentation losses. The panel places visual evidence and interpretation side by side without forcing the figure to continue onto the next page.



Failure mode. The error overlays make the boundary problem more explicit than the raw masks. The model usually captures the pet body, but red error regions remain around object contours and thin structures. This explains why Focal Loss did not solve the problem: many hard pixels are ambiguous boundary pixels, so simply increasing their weight can emphasize uncertainty rather than produce a cleaner contour.

Figure 12: Segmentation failure cases with compact text integration. The most persistent errors occur at object boundaries, thin structures, and low-contrast regions rather than in the coarse pet/background separation.

5 Task 2: VisDrone Detection and Tracking

5.1 VisDrone Data Preparation

The detection task uses the public VisDrone2019-DET dataset rather than a substitute dataset. The raw train, validation, and test-dev splits were downloaded and converted into YOLO format. The converted dataset contains 6471 training images, 548 validation images, and 1610 test-dev images. The training labels contain 343205 valid boxes after ignoring VisDrone non-target categories. The 10 object classes are pedestrian, people, bicycle, car, van, truck, tricycle, awning-tricycle, bus, and motor.

VisDrone is substantially harder than ordinary object detection datasets because many targets are small, partially occluded, densely packed, or viewed from high altitude. Therefore, modest mAP values are expected, especially for lightweight detectors at 640-pixel resolution.

5.2 Detector Training Results

Table 5: VisDrone detector comparison.

Model	Size	Epoch	Prec.	Rec.	mAP50-95
YOLOv8n	640	48	0.4188	0.3226	0.1665
YOLOv8s	640	47	0.5187	0.3922	0.2211
YOLO11m	960	36	0.6240	0.5257	0.3310

YOLOv8n is the required lightweight one-stage baseline. It reached 0.2985 mAP50 and 0.1665 mAP50-95. YOLOv8s improved mAP50-95 to 0.2211, showing that extra capacity helps in dense aerial scenes. The strongest extension was YOLO11m trained at image size 960. Even though the run was manually stopped at epoch 36 because of deadline constraints, it reached 0.5356 mAP50 and 0.3310 mAP50-95, clearly outperforming YOLOv8s.

The improvement has two sources that cannot be perfectly separated in this experiment: the newer YOLO11m architecture and the larger 960-pixel input size. Both are plausible contributors. Small objects occupy very few pixels in VisDrone, so higher resolution directly improves feature visibility. At the same time, the larger model has more capacity to represent dense object patterns and difficult class distinctions. The practical conclusion is that for VisDrone, a stronger detector at higher resolution is much more useful for downstream tracking than a very small detector, as long as the available GPU memory permits it.

5.3 Video Tracking Setup

The trained YOLO11m detector was used for a 30-second tracking and counting experiment. The original uploaded video was 1280×720 at 30 FPS and 41.2 seconds long. The final clip uses the 10s–40s segment, giving exactly 900 frames. The scene contains rainy road traffic, pedestrians, bicycles, e-bikes, motorcycles, cars, umbrellas, reflections, dense crossing motion, and partial occlusions. This makes it more challenging and more realistic than a sparse clean-weather video.

The virtual counting line is placed from $(250, 520)$ to $(1020, 520)$ across the lower road region. A crossing is counted when the center of a tracked bounding box changes sign relative to this line. Each track ID is counted at most once, which prevents repeated counting of a stable track. However, if an ID switch occurs before and after the line, duplicate counting can still happen. The current logic counts total crossings and does not separate direction.

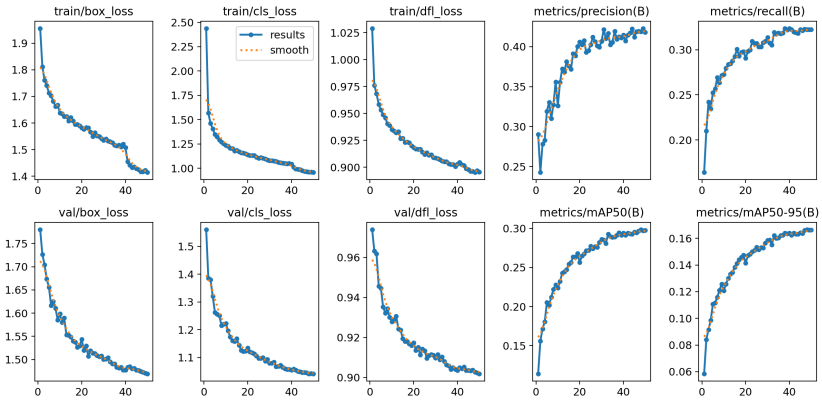
Table 6: Tracking and line-crossing results on the 30-second video.

Tracker	Conf.	Unique IDs	Avg. Obj.	Count
ByteTrack	0.25	183	5.79	11
BoT-SORT	0.25	177	5.90	8
ByteTrack	0.50	86	2.30	5

5.4 Tracking Results and Interpretation

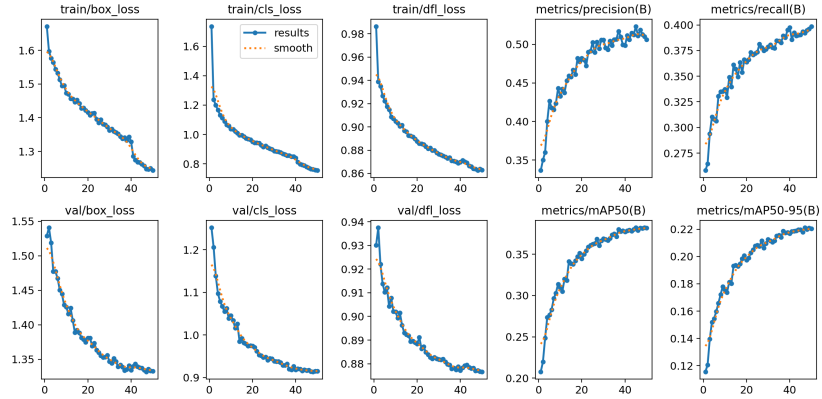
Table 6 shows that ByteTrack at confidence 0.25 produced the main result, with 183 unique track IDs, an average of 5.79 tracked objects per frame, and 11 line crossings. BoT-SORT at the same threshold was more conservative and counted 8 crossings. ByteTrack at confidence 0.50 counted only 5 crossings because many weak detections were suppressed.

This comparison illustrates a precision-recall trade-off at the tracking level. A lower confidence threshold keeps weak detections and reduces missed crossings, but it can also introduce short-lived tracks and more opportunities for ID fragmentation. A higher threshold produces cleaner detections but misses small, blurred, or occluded road users, which directly causes under-counting. In this video, the 0.25 threshold is more appropriate because the scene contains rain, small targets, and partial occlusion.



Baseline reading. YOLOv8n improves steadily, but recall remains low for VisDrone. This is expected because many pedestrians and vehicles occupy very few pixels, and dense occlusion makes localization difficult. The lightweight model is useful as a required baseline, but it is not strong enough for reliable downstream counting.

Figure 13: YOLOv8n training summary on VisDrone with adjacent interpretation.



Capacity effect. Compared with YOLOv8n, YOLOv8s improves precision, recall, mAP50, and mAP50-95. This shows that the low baseline score is not only a training failure; VisDrone genuinely benefits from larger detector capacity. The trade-off is model size and training time, which matters because the detector is later used for video tracking.

Figure 14: YOLOv8s training summary on VisDrone with compact explanation.

The difference between ByteTrack and BoT-SORT should not be simplified as one being universally better. The two trackers made different association decisions in dense traffic. ByteTrack preserved more crossing candidates and therefore produced a higher count. BoT-SORT was more conservative in the final count. Without manually annotated video ground truth, the count should be treated as a reproducible estimate rather than an absolute traffic measurement.

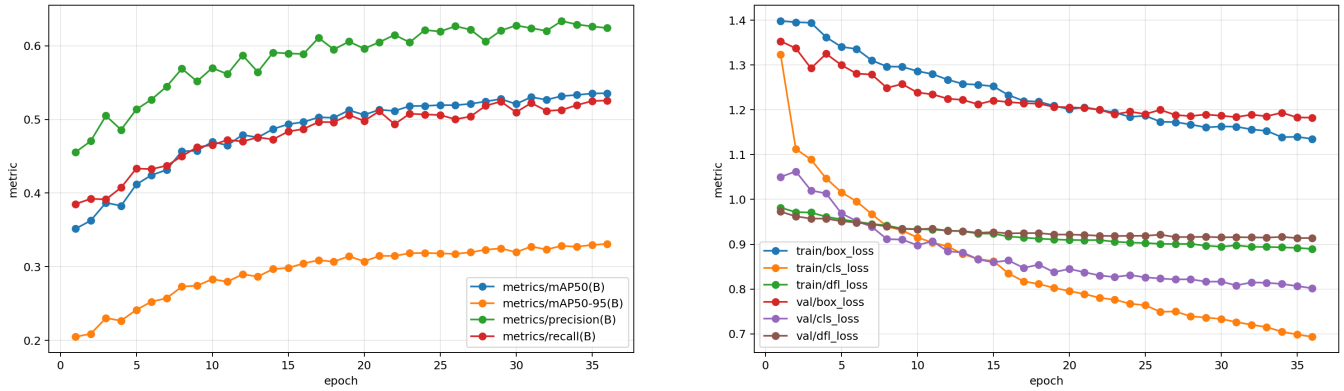


Figure 15: YOLO11m validation metrics and losses on VisDrone. The manually truncated run still achieved the best recorded score at epoch 36, while the losses were still decreasing slowly, suggesting possible remaining improvement with more epochs.

Figure 16: Four consecutive dense and occluded tracking frames. The scene contains overlapping pedestrians, bicycles or e-bikes, vehicles, rain, and small objects, making ID preservation difficult.

Figure 17: A consecutive line-crossing sequence. The tracked object crosses the virtual line and the count changes from 2 to 3 around frame 632.

Counting snapshot. This later frame shows the accumulated count after several crossing events. It also exposes the limitation of the pipeline: some visible road users remain difficult under rain, distance, and occlusion. Thus the number should be treated as a reproducible algorithmic estimate rather than a manually verified traffic statistic.

Figure 18: A later tracking snapshot with accumulated line count and adjacent interpretation.

5.5 Counting Error Analysis

The line-crossing module is intentionally simple and transparent. Its main strength is that it uses persistent track IDs, so a stable object is not counted repeatedly after crossing the line. Its main weakness is that it inherits all detector and tracker errors. Under-counting occurs when a target is not detected near the line, when confidence thresholding suppresses it, or when the track disappears before the sign change is observed. Duplicate counting can occur if one physical object receives a new ID after an occlusion and crosses the line again under that new ID. Perspective also matters: a 2D image line is only an approximation of a real-world road boundary, especially when the camera is high and tilted.

Rain and wet-road reflections made this video a better stress test than a clean static scene. The final results are therefore realistic but imperfect. They demonstrate the full required pipeline–detection, tracking IDs, occlusion frames, and line counting–while also making clear that reliable traffic statistics would require annotated video ground truth, camera calibration, and direction-aware counting.

6 Cross-Task Reflection

Across the three tasks, the most consistent lesson is that stronger methods help only when they match the dominant bottleneck. In classification, the bottleneck is fine-grained representation, so ImageNet pretraining and moderate depth are useful. Adding SE attention without enough evidence did not help. In segmentation, the bottleneck is region overlap and boundary quality, so Dice loss improves mIoU, but Focal Loss does not solve ambiguous trimap boundaries. In VisDrone detection, the bottleneck is small-object visibility and dense localization, so a larger detector and higher input resolution substantially improve mAP and downstream tracking.

The negative results are as important as the positive ones. Random initialization was far worse than pretraining; SE attention did not beat the simpler baseline; Focal Loss did not beat Dice; and a higher tracking confidence threshold reduced the crossing count. These results prevent the report from becoming a list of successful tricks. They show that each modification must be judged by the task metric, qualitative evidence, and failure mode.

7 Conclusion

This project completed the required image recognition, semantic segmentation, object detection, video tracking, occlusion analysis, and line-crossing counting tasks using a unified reproducible harness. The best classification model was a pretrained ResNet-34 with 90.60% test accuracy. The best required segmentation model was a from-scratch U-Net with Dice loss, reaching 77.90% test mIoU. The strongest VisDrone detector was YOLO11m at 960-pixel resolution, reaching 0.5356 mAP50 and 0.3310 mAP50-95 before manual truncation at epoch 36. The tracking stage showed that ByteTrack with a lower confidence threshold produced the most complete crossing count, while higher thresholds under-counted difficult objects.

The main limitation is that some extensions were constrained by time and compute. The YOLO11m run may have improved with a full 50-epoch schedule, and the video counting result lacks manually annotated ground truth. Nevertheless, the experiments provide quantitative comparisons, visual evidence, and failure analysis for all required components, while preserving reproducibility through the harness and configuration files.

References

[1] O. M. Parkhi, A. Vedaldi, A. Zisserman, and C. V. Jawahar. Cats and Dogs. In *IEEE Conference on Computer Vision and Pattern Recognition*, 2012.

[2] K. He, X. Zhang, S. Ren, and J. Sun. Deep Residual Learning for Image Recognition. In *IEEE Conference on Computer Vision and Pattern Recognition*, 2016.

[3] O. Ronneberger, P. Fischer, and T. Brox. U-Net: Convolutional Networks for Biomedical Image Segmentation. In *MICCAI*, 2015.

[4] P. Zhu et al. Vision Meets Drones: A Challenge. *arXiv:1804.07437*, 2018.

[5] G. Jocher, A. Chaurasia, and J. Qiu. Ultralytics YOLO. https://github.com/ultralytics/ultralytics, 2023.

[6] Y. Zhang, P. Sun, Y. Jiang, D. Yu, F. Weng, Z. Yuan, P. Luo, W. Liu, and X. Wang. ByteTrack: Multi-Object Tracking by Associating Every Detection Box. In *ECCV*, 2022.